

Trust Nothing

**The failure mode with
no error message.**

Your loop runs clean and you have no idea whether any of it is true. Today it learns to look at its own output and refuse to publish. Twenty minutes on one idea, plus a few where this thing finally gets a face.

Everything works. That's the problem.

Real listings, deterministic scores, a ranked gap report. It runs clean. Nothing throws. Every number is arithmetically correct — and computed from facts a language model extracted days ago, unsupervised.

IF EXTRACTION QUIETLY MISSED “K8S” ON 40 LISTINGS

Your gap report is **confidently, precisely wrong**. No exception. No stack trace. A beautiful table telling you to learn the wrong thing.

Unattended is not autonomous.

UNATTENDED

- 1 Runs without you
- 2 You have that today

AUTONOMOUS

- 1 Runs without you **and** you can trust the output
- 2 One capability short

THE GAP BETWEEN THOSE TWO WORDS

The system has to be able to look at its own output and say **no**. That's the last hard problem in this project.

What you built today

- 1 Four **plausibility checks** as pure functions, with tests.
- 2 A Verifier that **judges and never repairs** — the Orchestrator owns the fix.
- 3 One **capped retry** with adjusted context, then the cycle is marked degraded.
- 4 Verdicts logged with the failing check **and the observed value**.
- 5 A failed cycle **can never overwrite good data** — stale beats wrong.
- 6 And the **Agent Activity Log** on a screen — panel three of three, from session one.

Plausibility, never correctness. **There is no ground truth for a fit score** — and you don't need one to catch the failures that actually happen.

Your Verifier, running

The build

- 1 All four checks, with tests
- 2 Thresholds in config, **each with a comment**
- 3 Retry capped at **one** — I look for this
- 4 Verdicts log the observed value

And the dashboard

- 1 **streamlit run app.py** works
- 2 Panels read the **last passing cycle** only
- 3 Activity log shows the failures
- 4 No “run cycle” button anywhere

Make it reject something.

Flatten your scores, run the cycle, and film the rejection in your activity log. **Both outcomes count.**

THE PAYOFF OF THE CLASS

```
$ python run_cycle.py
```

```
scorer    scored 50 · range 71-75 · mean 73
```

VERDICT: fail — score_spread observed 4 (min 10)

```
orchestr. retry 1/1 · scorer, stricter rubric
```

VERDICT: pass — spread 31-89

...or: cycle DEGRADED · not published

Third week I've asked you to film a failure. That's deliberate — **the working state is easy and proves nothing.** What proves your system works is watching it refuse.

DO THIS IN THE NEXT HOUR · FIVE MINUTES

Leave it running. Then break something.

Keep the dashboard open in a browser tab. Flatten a weight, run the cycle, refresh, and watch the rejection appear in the log in front of you.

THAT LOOP IS HOW THE THRESHOLDS BECOME YOURS

Then post it — **#MyEdge**, a screenshot of the log with a rejection in it. “My agent refused to trust itself this morning” beats a green tick.

Also in **prompts.md**: C6-P5 builds a proper **python -m edgedash.verdicts** view. Ten minutes, not covered in this video.

Your system can say no.

Next week it runs on a schedule without you touching it, and then it goes public with a real URL. The dashboard you just built is the thing that gets deployed.

But today was the one that mattered. Everything before this was a program that produced output. Now it's a system with a standard.

Break something. Film the rejection. See you next week.